on-off control

November 24, 2025

0.1 Import

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
import tclab
import time
from IPython.display import display, clear_output
```

[ ]:
```

```

0.2 Parametri simulazione

[2]:
```
n = 180  # Number of second time points
tm = np.linspace(0,n-1,n) # Time values
T1 = np.zeros(n) # vector of measured temperatures
Q1 = np.zeros(n) # vector of actuated controls


# step setpoint from 23.0 to 40.0 degC
set_point= 40.0
SP1 = np.ones(n)*23.0 # 23 is assumed as air temperature and reference value␣
 ↪for the first 10 seconds
SP1[10:] = set_point
Q1_bias = 0.0
```

0.3 Avvio simulazione

[3]:
```
lab = tclab.TCLab()
channel= 1   #select which heater and sensor to use, use 2 to speed up the␣
 ↪lecture and avoid cooling
```

```
TCLab version 1.0.0
Arduino Leonardo connected on port COM5 at 115200 baud.
TCLab Firmware 2.0.1 Arduino Leonardo/Micro.
```

[5]:
```
print('T1=', lab.T1, ' T2=',lab.T2) # check initial conditions
```

```
T1= 28.633  T2= 25.732
```

0.3.1 Loop simulazione e plot

[6]:
```
# Start live plot
plt.ion()# Turn on matplotlib interactive mode
fig, axs = plt.subplots(2, 1, figsize=(20, 14))



# simulation loop

for i in range(n): # for all time steps


    # record measurement
    if channel==1:
        T1[i] = lab.T1
    else:
        T1[i] = lab.T2


    # On-off controller
    if T1[i]<SP1[i]:
        Q1[i]=100
    elif T1[i]>SP1[i]:
        Q1[i]=0

    else: #only for code stability
        Q1[i]=0

    # Actuate control
    if channel==1:
        lab.Q1(Q1[i])
    else:
        lab.Q2(Q1[i])

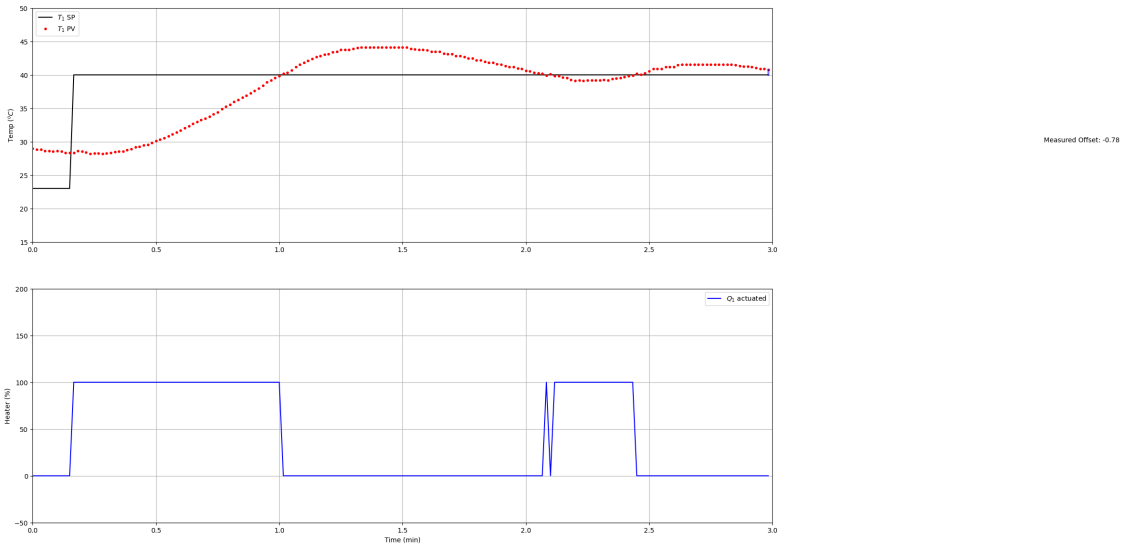
    # --- Live updating plot ---
    clear_output(wait=True)
    axs[0].cla()
    axs[1].cla()
    axs[0].grid()
    axs[0].plot(tm[:i+1]/60.0, SP1[:i+1], 'k-', label=r'$T_1$ SP')
    axs[0].plot(tm[:i+1]/60.0, T1[:i+1], 'r.', label=r'$T_1$ PV')
    axs[0].text(4.1, 30, 'Measured Offset: ' + str(np.round(SP1[i]-T1[i],2)))
    axs[0].plot([tm[i]/60.0, tm[i]/60.0], [SP1[i], T1[i]], 'b-', lw=3, alpha=0.
↪5)
```

```
    axs[0].set_ylabel(r'Temp ($^oC$)')
    axs[0].set_xlim([0, n/60])
    axs[0].set_ylim([15, np.max(SP1)*1.25])
    axs[0].legend(loc=2)

    axs[1].grid()
    axs[1].plot(tm[:i+1]/60.0, Q1[:i+1], 'b-', label=r'$Q_1$ actuated')

    axs[1].set_ylabel(r'Heater (%)')
    axs[1].set_xlabel('Time (min)')
    axs[1].legend(loc=1)
    axs[1].set_xlim([0, n/60])
    axs[1].set_ylim([-50,200])
#    fig.tight_layout()
    display(fig)
    plt.pause(0.01)
    time.sleep(1)

plt.ioff()  # Turn off interactive mode
```




```
TCLab disconnected successfully.
```

0.3.2 Comportamento oscillatorio, sbilanciato verso l'alto per la natura termica del sistema

0.3.3 Interrompi simulazione

[15]:
```
lab.close()
```

```
TCLab disconnected successfully.
```

0.3.4 Secondo test, aggiungiamo delle bande di tolleranza

[20]:
```
n = 180  # Number of second time points
tm = np.linspace(0,n-1,n) # Time values
T1 = np.zeros(n) # vector of measured temperatures
Q1 = np.zeros(n) # vector of actuated controls


# step setpoint from 23.0 to 40.0 degC
set_point= 40.0
SP1 = np.ones(n)*23.0 # 23 is assumed as air temperature and reference value␣
 ↪for the first 10 seconds
SP1[10:] = set_point
Q1_bias = 0.0


threshold_up = SP1 +2.5 # upper and lower temperature bounds to activate heater
threshold_down= SP1-2.5
```

[ ]:
```
```

[19]:
```
lab = tclab.TCLab()
print('T1=', lab.T1, ' T2=',lab.T2) # check initial conditions
channel= 2   #select which heater and sensor to use, use 2 to speed up the␣
 ↪lecture and avoid cooling
```

```
TCLab version 1.0.0
Arduino Leonardo connected on port COM5 at 115200 baud.
TCLab Firmware 2.0.1 Arduino Leonardo/Micro.
T1= 21.865  T2= 22.188
```

[21]:
```
# Start live plot
plt.ion()# Turn on matplotlib interactive mode
fig, axs = plt.subplots(2, 1, figsize=(20, 14))



# simulation loop

for i in range(n): # for all time steps


    # record measurement
    if channel==1:
        T1[i] = lab.T1
    else:
        T1[i] = lab.T2
```

```
    # On-off controller
    if T1[i]<threshold_down[i]:
        Q1[i]=100
    elif T1[i]>threshold_up[i]:
        Q1[i]=0
    else: #only for code stability
        Q1[i]=0

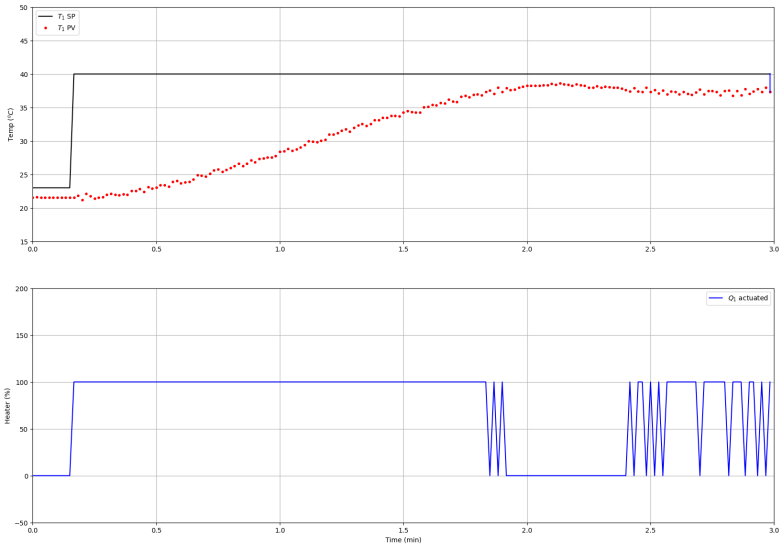
    # Actuate control
    if channel==1:
        lab.Q1(Q1[i])
    else:
        lab.Q2(Q1[i])

    # --- Live updating plot ---
    clear_output(wait=True)
    axs[0].cla()
    axs[1].cla()
    axs[0].grid()
    axs[0].plot(tm[:i+1]/60.0, SP1[:i+1], 'k-', label=r'$T_1$ SP')
    axs[0].plot(tm[:i+1]/60.0, T1[:i+1], 'r.', label=r'$T_1$ PV')
    axs[0].text(4.1, 30, 'Measured Offset: ' + str(np.round(SP1[i]-T1[i],2)))
    axs[0].plot([tm[i]/60.0, tm[i]/60.0], [SP1[i], T1[i]], 'b-', lw=3, alpha=0.
 ↪5)
    axs[0].set_ylabel(r'Temp ($^oC$)')
    axs[0].set_xlim([0, n/60])
    axs[0].set_ylim([15, np.max(SP1)*1.25])
    axs[0].legend(loc=2)

    axs[1].grid()
    axs[1].plot(tm[:i+1]/60.0, Q1[:i+1], 'b-', label=r'$Q_1$ actuated')

    axs[1].set_ylabel(r'Heater (%)')
    axs[1].set_xlabel('Time (min)')
    axs[1].legend(loc=1)
    axs[1].set_xlim([0, n/60])
    axs[1].set_ylim([-50,200])
#    fig.tight_layout()
    display(fig)
    plt.pause(0.01)
    time.sleep(1)

plt.ioff()  # Turn off interactive mode
lab.close()
```



Measured Offset: 2.67

```
TCLab disconnected successfully.
```

0.3.5 Lo sbilanciamento è ora verso il basso ma questo controllore fallisce il task di tracking

[ ]:

[ ]: